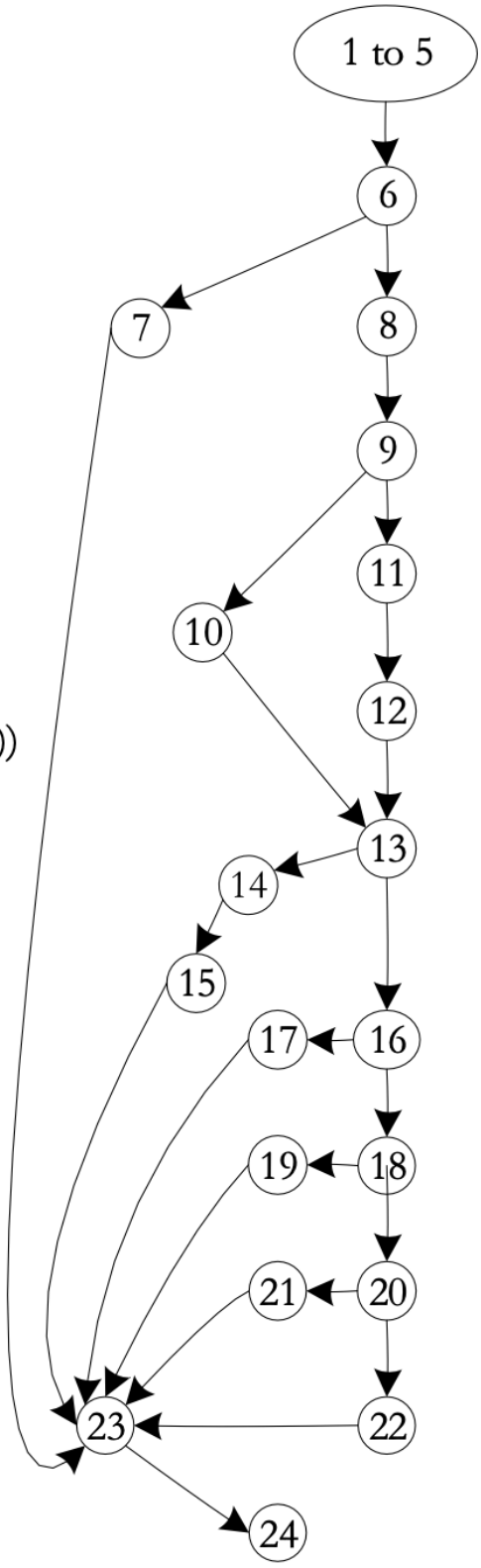
DD Path

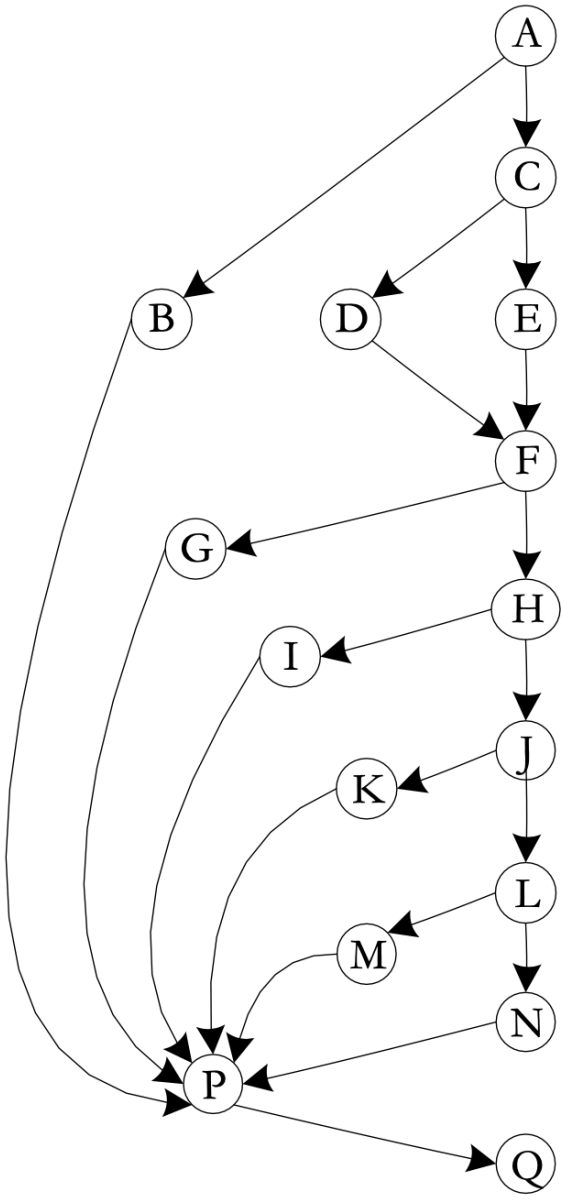
Given a program written in an imperative language, its ***DD-Path graph*** is the directed graph in which nodes are DD-Paths of its program graph, and edges represent control flow between successor DD-Paths.

```
1   public static int triangle3(int a, int b, int c) {
1
1     boolean c1, c2, c3, isATriangle;
1
1     // Step 1: Validate Input
2       c1 = (1 <= a) && (a <= 300);
3       c2 = (1 <= b) && (b <= 300);
4       c3 = (1 <= c) && (c <= 300);
5
5       int triangleType = INVALID;
6       if(!c1 || !c2 || !c3)
7         triangleType = OUT_OF_RANGE;
8       else {
8
8       // Step 2: Is A Triangle?
9         if((a < b + c) && (b < a + c) && (c < a + b))
10          isATriangle = true;
11        else
12          isATriangle = false;
12
12      // Step 3: Determine Triangle Type
13        if(isATriangle) {
14          if((a == b) && (b == c))
15            triangleType = EQUILATERAL;
16          else if((a != b) && (a != c) && (b != c))
17            triangleType = SCALENE;
18          else
19            triangleType = ISOSELES;
20        } else
21          triangleType = INVALID;
22      }
23
23      return triangleType;
24    }
```



DD-Path Graph of Figure 8.2

Figure 8.2	*DD-Path*
1 - 6	**A**
7	**B**
8, 9	**C**
10	**D**
11, 12	**E**
13	**F**
14, 15	**G**
16	**H**
17	**I**
18	**J**
19	**K**
20	**L**
21	**M**
22	**N**
23	**P**
24	**Q**



1. Cyclomatic Complexity ($V(G)$)

- **Triangle Problem Program Graph:** $V(G) = 7$
- **Triangle Problem DD-Path Graph:** $V(G) = 7$
- **Calculation:** $V(G) = \text{Number of Predicate Nodes} + 1$.
  - The predicate nodes in the DD-Path graph are **A, C, F, H, J, and L** (nodes with an out-degree of 2).
  - $6 + 1 = 7$.

Are the complexities equal?

Yes. The cyclomatic complexity of a Program Graph and its corresponding DD-Path Graph are always **equal**.

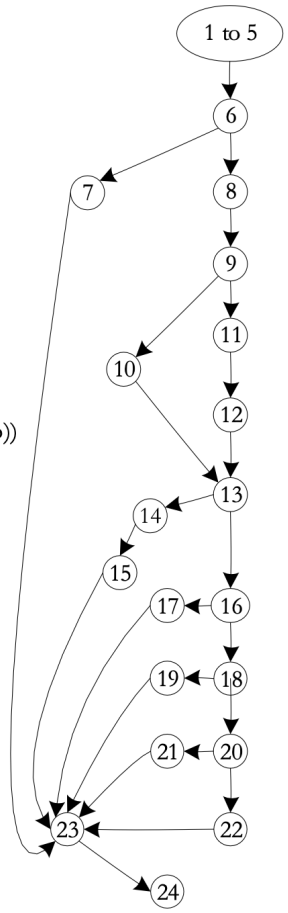
What conclusions can you draw?

- **Logic Preservation:** The DD-Path graph is a structural simplification that preserves the original decision logic and control flow of the program.
- **Complexity Invariance:** Condensing sequential statements into a single node does not change the number of linearly independent paths ($V(G)$).
- **Testing Requirement:** Both graphs indicate that exactly **7** independent paths must be tested to achieve full basis path coverage.

Statement about 2-connected components

A **DD-Path Graph** is a **Condensation Graph** of the Program Graph. In a Program Graph, **2-connected components** (which are maximal chains of nodes where every internal node has an in-degree of 1 and an out-degree of 1) are collapsed into single nodes to form the DD-Path Graph. Therefore, each node in a DD-Path Graph represents a 2-connected component from the original Program Graph.

```
1  public static int triangle3(int a, int b, int c) {
1
1    boolean c1, c2, c3, isATriangle;
1
1    // Step 1: Validate Input
2      c1 = (1 <= a) && (a <= 300);
3      c2 = (1 <= b) && (b <= 300);
4      c3 = (1 <= c) && (c <= 300);
5
5      int triangleType = INVALID;
6      if(!c1 | | !c2 | | !c3)
7        triangleType = OUT_OF_RANGE;
8      else {
8
8      // Step 2: Is A Triangle?
9        if((a < b + c) && (b < a + c) && (c < a + b))
10         isATriangle = true;
11       else
12         isATriangle = false;
12
12     // Step 3: Determine Triangle Type
13       if(isATriangle) {
14         if((a == b) && (b == c))
15           triangleType = EQUILATERAL;
16         else if((a != b) && (a != c) && (b != c))
17           triangleType = SCALENE;
18         else
19           triangleType = ISOSELES;
20       } else
21         triangleType = INVALID;
22     }
23
23     return triangleType;
24   }
```



Dependent DD-Paths

(Figure 8.2 redrawn)

Dependent DD-Paths can correspond to feasible and infeasible paths.

Feasible Paths in Figure 8.2

- If a path traverses node 10, it must traverse nodes 14 – 19
- If a path traverses node 12, it must traverse nodes 20 and 21

Infeasible Paths in Figure 8.2

- If a path traverses node 10, it cannot traverse nodes 20 and 21
- If a path traverses node 12, it cannot traverse nodes 14 - 19